

Assignment 2 (Week 4): Expression Conversion, Recursion, and Tower of Hanoi

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 9, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

Answer every question. Show all working for Numerical and Design questions. Each problem states its own assumptions; everything you need is in Week 4's Notes chapter alone. Notation matches the lecture exactly: **push/pop/peek**, **top** = -1 means empty, precedence $\wedge > * / > + -$, $\wedge$ is right-associative, all others left-associative, $T(n)$ counts Hanoi disk moves, d is runtime-stack depth.

4.1 Conceptual

Q1. Postfix and prefix need no parentheses and no precedence rules — position alone fixes the order (standard treatment; see Horowitz & Sahni Ch. 3) [1].

- (a) In one or two sentences, explain why infix needs all three rules (precedence, associativity, parentheses) while postfix does not — name what a single left-to-right scan of infix cannot resolve without remembering.
- (b) Group $x/y*z$ both ways: $(x/y) * z$ and $x/(y * z)$. With $x = 12$, $y = 3$, $z = 2$, compute both values and state which one the lecture's left-associativity rule picks.
- (c) Group $x\wedge y\wedge z$ both ways: $x^{(y^z)}$ and $(x^y)^z$. With $x = 2$, $y = 3$, $z = 2$, compute both values and state which one the lecture's right-associativity rule picks.

Why this matters: the compiler rewrites what you type before it ever runs it — this question pins down why the rewrite is needed at all, and why getting the tie-break wrong silently computes a different number.

Q2. A recursive function solves a smaller instance of its own problem: one **base case** answered directly, one **recursive case** that calls itself on strictly smaller input (Karumanchi Ch. 2, pp.62–73 PDF; Wirth Ch. 3) [2, 3]. Each call pushes a **frame** (locals, return address); each return pops it. **Iteration** keeps one frame and updates variables; **recursion** spends one frame per level, i.e. $O(d)$ auxiliary space for depth d .

- (a) In one or two sentences, explain the difference between the *number of moves* $2^n - 1$ and the *maximum live frames* n for Hanoi — what does each one count?
- (b) In one or two sentences, explain why a recursion depth of $n = 100,000$ bursts the runtime stack while a loop with 100,000 iterations does not (name who pays per level).

- (c) Name the two failure modes from the lecture besides deep overflow that make recursion the wrong tool (repeated subproblems, missing base case), one line each, and state Wirth's rule of thumb for when to prefer iteration (Ch. 3, pp.87–108).

Why this matters: recursion and iteration have the same power but different ledgers — this question checks you can read the bill (frames alive now) separately from the receipt (calls ever made).

4.2 Numerical

Q3. Convert the infix string $p-q*(r+s)/t^u^v$ to postfix with the lecture's one-stack algorithm (Karumanchi Ch. 4, pp.163–204 PDF) [2]: operands append to output; $($ pushes; $)$ pops to output until $($ (both parens discarded); operator op pops waiting operators of $\geq$ precedence (strictly $>$ if op is right-associative), then pushes.

- (a) Give the output string and the stack contents after *every* token, as a small table with one row per token (15 tokens: $p, -, q, *, (, r, +, s,), /, t, ^, u, ^, v$).
- (b) State the final postfix string (no spaces).
- (c) Name the two tokens where the lecture's traps bite in this string: the token that must pop a waiting operator of *equal* precedence (left-associativity), and the token that must *not* pop a waiting operator of equal precedence (right-associativity) — one sentence each, naming the operators involved.

Why this matters: one pass with each token pushed and popped at most once — $O(n)$ time, $O(n)$ space — but only if the tie-break fires in the right direction at exactly these two spots.

Q4. Two pricings: a conversion and a recurrence.

- (a) Convert the prefix string $-+*ab+cde$ to postfix by the lecture's right-to-left scan (operand: push as a string; operator op : $a = \text{pop}()$, $b = \text{pop}()$, push $a + " " + b + " " + op$). Give the stack contents after *every* token and the final postfix string.
- (b) For Tower of Hanoi with $T(1) = 1$ and $T(n) = 2T(n-1) + 1$ ($n > 1$): unfold the recurrence for $n = 6$ step by step from $T(5) = 31$, state the move count $T(6)$ and the maximum live frames, then state $T(10)$. For each bound, name the operation, give the bound, and state the case (Week 2 shape).
- (c) At one billion moves per second, how many years do the $2^{64} - 1$ moves for $n = 64$ take? Show the division chain (seconds $\rightarrow$ years) and give the answer to the nearest year.

Why this matters: conversion costs a linear scan while Hanoi costs an exponential one — this question makes you feel both prices in concrete numbers on the same page.

4.3 Design

Q5. Convert the infix string $m*(n+p)-q/r$ to *prefix* by the lecture's reversal route: (1) reverse the string and swap $($ with $)$; (2) run the postfix algorithm under the mirror rule (pop only on *strictly greater* precedence); (3) reverse the result.

- (a) Give the reversed-and-swapped string.

- (b) Give the postfix of that reversed string, showing the output and stack after every token.
- (c) Reverse it back to the final prefix string, then sanity-check it outside-in: read the outermost operator first and confirm it says “subtract q/r from $m \times (n + p)$.” State the check in one or two sentences.

Why this matters: prefix is the mirror of postfix on the reversed string — one algorithm serves both directions, but only if the paren swap and the mirror rule both fire; this question makes you execute both on a string the lecture never worked.

Q6. The runtime stack as a design constraint. Recall the lecture’s canonical recursion:

```
int fact(int n) {
    if (n <= 1) return 1;          /* base case */
    return n * fact(n - 1);        /* smaller instance */
}
```

- (a) Trace the live-frame depth of `fact(4)`: list each running call from `main` down to the deepest frame, and state the maximum frames alive at once (count `main`’s frame).
- (b) A classmate writes `int down(int n) { return n * down(n - 1); }` with no base case and reports “it crashes on large n but works on small n .” In two or three sentences, explain what actually happens on *every* input (including small n): which lecture failure mode this is, and why no input size escapes it.
- (c) Choose recursion vs. iteration with one-sentence Wirth-style justification each: (i) moving a 20-disk Hanoi tower (self-similar, depth 20); (ii) summing 100,000 array elements with a plain loop available. Name the depth/space ledger that decides each choice.

Why this matters: depth is the longest chain alive, and the stack only holds so many frames — this question turns that picture into two shipping decisions, one where recursion earns its frames and one where it does not.

References

- [1] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Books, 2nd edition, 2008.
- [2] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1’s C-specific pointer/malloc content — see this course’s supporting-books index for the coverage check.
- [3] Niklaus Wirth. *Algorithms and Data Structures*. Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.